

```
"""
geometry.py – Acoustic sphere geometry. Everything derived from Bessel zeros.
```

```
The sphere is given. Everything else is solved from it outward.
```

```
"""
```

```
import math
import numpy as np
from scipy.special import spherical_jn
from scipy.optimize import brentq
from dataclasses import dataclass
from typing import List, Tuple
```

```
def bessel_zeros(n: int, n_zeros: int = 5) -> List[float]:
```

```
    """
```

```
    Find the first n_zeros zeros of spherical Bessel function j_n(x).
```

```
    These are exact – not approximated.
```

```
    """
```

```
    zeros = []
```

```
    # Search in windows of pi starting from (n+1)*pi/2
```

```
    x_start = max(0.1, (n + 0.5) * math.pi * 0.5)
```

```
    x = x_start
```

```
    dx = 0.1
```

```
    while len(zeros) < n_zeros:
```

```
        # Find sign change
```

```
        while spherical_jn(n, x) * spherical_jn(n, x + dx) > 0:
```

```
            x += dx
```

```
        zero = brentq(lambda t: spherical_jn(n, t), x, x + dx, xtol=1e-12)
```

```
        zeros.append(zero)
```

```
        x += dx
```

```
    return zeros
```

```
@dataclass
```

```
class SphericalMode:
```

```
    """A single resonant mode of the acoustic sphere."""
```

```
    n: int          # radial order
```

```
    l: int          # angular order
```

```
    bessel_zero: float # j_nl zero value
```

```
    k: float        # wavenumber = j_nl / R
```

```
    f: float        # resonant frequency = c * k / (2*pi)
```

```
    wavelength: float # lambda = c / f
```

```

@dataclass
class SystemGeometry:
    """
    Complete system geometry derived from sphere outward.
    Every value is exact – derived, not chosen.
    """
    # Sphere
    L: float          # box side length (m)
    R: float          # sphere radius = L/2
    mode: SphericalMode # resonant mode

    # Pendulum
    L_pendulum: float  # pendulum length (m)
    T_pendulum: float  # pendulum period (s)
    f_pendulum: float  # pendulum frequency (Hz)

    # Venturi
    r0: float          # throat radius (m)
    r_in: float         # inner cutoff (m)
    G: float           # Bernoulli gain

    # Blind gate
    d_slit: float       # slit spacing (m)
    theta_diffraction: float # first-order angle (rad)

    # Lens
    f_lens: float       # focal length (m)

    # Fan / interference
    f_fan: float        # fan frequency (Hz) – exact
    T_star: float       # stellar temperature this system is resonant with (K)
    nu_star: float      # resonant photon frequency (Hz)

    def summary(self):
        print("=== ACOUSTIC SPHERE SYSTEM GEOMETRY ===")
        print(f"  Box side L:          {self.L:.4f} m")
        print(f"  Sphere radius R:     {self.R:.4f} m")
        print(f"  Mode (n,l):          ({self.mode.n}, {self.mode.l})")
        print(f"  Bessel zero j_n1:    {self.mode.bessel_zero:.6f}")
        print(f"  Resonant freq:       {self.mode.f:.4f} Hz")
        print()
        print(f"  Pendulum length:     {self.L_pendulum:.6f} m")
        print(f"  Pendulum period:     {self.T_pendulum:.6f} s")
        print()
        print(f"  Venturi r0:          {self.r0:.6f} m")
        print(f"  Venturi r_in:        {self.r_in:.6f} m")

```

```

print(f"  Bernoulli gain G:  {self.G:.6f}")
print()
print(f"  Blind slit d:        {self.d_slit:.6e} m")
print(f"  Diffraction angle: {math.degrees(self.theta_diffraction):.4f} d")
print()
print(f"  Fan frequency:        {self.f_fan:.6f} Hz")
print(f"  Stellar T:            {self.T_star:.2f} K")
print(f"  Resonant  $\nu$ :        {self.nu_star:.4e} Hz")

```

```

class AcousticSphereGeometry:
    """
    Derives complete system geometry from sphere outward.

    Usage::
        geo = AcousticSphereGeometry(L=1.0, n=0, l=0, mode_index=1)
        system = geo.solve()
        system.summary()
    """

    # Physical constants
    C_AIR = 343.0          # speed of sound (m/s)
    G_GRAV = 9.81          # gravity (m/s2)
    H = 6.626e-34          # Planck constant
    K_B = 1.381e-23        # Boltzmann constant
    C_LIGHT = 3e8          # speed of light (m/s)

    def __init__(self, L: float = 1.0, n: int = 0, l: int = 0,
                  mode_index: int = 1, D_blind: float = None):
        """
        L:                box side length (m)
        n, l:             spherical harmonic orders for the resonant mode
        mode_index:       which zero of  $j_n$  to use (1 = first zero)
        D_blind:          distance from blind gate to Venturi (default L/4)
        """
        self.L = L
        self.R = L / 2.0
        self.n = n
        self.l = l
        self.mode_index = mode_index
        self.D_blind = D_blind if D_blind else L / 4.0

    def solve(self) -> SystemGeometry:
        """
        Derive everything from the sphere outward.
        Returns complete SystemGeometry.
        """

```

```

# — 1. Sphere mode —————
zeros = bessell_zeros(self.n, self.mode_index + 1)
j_n1 = zeros[self.mode_index - 1]
k = j_n1 / self.R
f_mode = self.C_AIR * k / (2 * math.pi)
lam = self.C_AIR / f_mode

mode = SphericalMode(
    n=self.n, l=self.l,
    bessell_zero=j_n1, k=k, f=f_mode, wavelength=lam
)

# — 2. Pendulum —————
# T_pendulum = 1/f_mode → L_pend = g/(4π²f²)
T_pend = 1.0 / f_mode
L_pend = self.G_GRAV / (4 * math.pi**2 * f_mode**2)

# — 3. Venturi throat —————
# r0 = R * j_n1 / pi (exact from Bessel zero)
r0 = self.R * j_n1 / math.pi

# r_in from previous Bessel zero (n=0 case: use j_n1/2)
if self.mode_index > 1:
    j_prev = zeros[self.mode_index - 2]
    r_in = self.R * j_prev / math.pi
else:
    r_in = r0 * 0.25 # first mode: inner cutoff = r0/4

G = max(0.0, 1.0 - (r_in / r0)**2)

# — 4. Blind gate slit spacing —————
# d = lam * sqrt(r0² + D²) / r0 (first-order diffraction at r0)
theta = math.atan2(r0, self.D_blind)
d_slit = lam / math.sin(theta)

# — 5. Lens focal length —————
# f1 = r0² / lam (Rayleigh range)
f_lens = r0**2 / lam

# — 6. Fan frequency and stellar resonance —————
# From sphere: f_fan = c_air * j_n1 / (2pi * R) = f_mode
f_fan = f_mode

# From starlight: f_fan = hv / (kT * 2pi)
# -> T_star = hv / (kT * 2pi * f_fan)
# But v is the optical frequency – use visible light center ~550nm
# Actually: the fan frequency sets the MODULATION of the synthetic light

```

```

# The interference condition:  $\omega_{\text{fan}} = h\nu/kT$ 
# ->  $\nu = kT * \omega_{\text{fan}} / h = kT * 2\pi * f_{\text{fan}} / h$ 
nu_star = self.K_B * 2 * math.pi * f_fan / self.H
# -> T from Wien's law:  $T = h\nu / (k * x)$  where  $x \approx 2.821$ 
WIEN_X = 2.821
T_star = self.H * nu_star / (self.K_B * WIEN_X)

return SystemGeometry(
    L=self.L, R=self.R, mode=mode,
    L_pendulum=L_pend, T_pendulum=T_pend, f_pendulum=f_mode,
    r0=r0, r_in=r_in, G=G,
    d_slit=d_slit, theta_diffraction=theta,
    f_lens=f_lens,
    f_fan=f_fan, T_star=T_star, nu_star=nu_star,
)

```